



EL CARTEL DE LOS POLLOS

Documentación técnica del proyecto

Traspaso (handoff) para equipo de desarrollo externo

Sistema POS / PWA — venta, caja, inventario (recetas), cocina (KDS), reportes de gerencia, cartelería y catálogo público.

Generado: 04 de junio de 2026

Producción: cartel-pollos.onrender.com

Contenido

1. Guía de Handoff
2. Visión general (README)
 3. Arquitectura
 4. Módulos
5. Modelo de datos
 6. API
7. Seguridad y RBAC
 8. Matriz RBAC
9. Eventos de auditoría
 10. Onboarding
11. Notas de migración
12. Addendum: Código y Lógica de Negocio

Guía de Handoff (traspaso a equipo externo)

Resumen del producto

PWA de punto de venta (POS) para rotisería de pollos en Chile (es-CL). Incluye venta, caja con cierre ciego, despacho/KDS de cocina, inventario con recetas (BOM), mermas, reposición con costo, reportes de gerencia (P&L, flujo, estadísticas, predicción de horno), cartelería pública para TV y catálogo público.

Stack

- Monorepo npm workspaces: apps/backend (Node + Express, ESM) y apps/frontend (Vite + React + Tailwind, PWA).
- Base de datos: Turso / libSQL (@libsql/client). SQL en apps/backend/db/schema.sql.
- Un mismo servicio sirve la API (/api) y la PWA (dist) en el mismo dominio.
- Auth JWT HS256; HMAC por sesión para firmar ventas; TOTP para mutaciones sensibles; PIN admin (bcrypt) para ajustes de stock; RBAC por matriz role_permissions; audit_logs append-only.

Repositorios y despliegue

- Producción: <https://cartel-pollos.onrender.com> (Render, autoDeploy desde rama main).
- Deploy: push a main -> Render reconstruye y publica. Config en render.yaml.
- Datos: Turso (token en .env.production, NO versionado).

Variables de entorno (no se versionan)

- backend .env / .env.production: TURSO_DATABASE_URL, TURSO_AUTH_TOKEN, JWT_SECRET (y afines).
- El frontend usa el mismo origen para /api (sin URL embebida).

Comandos

- Instalar: npm install (en la raíz).
- Test backend: npm test --workspace @cartel/backend (deben quedar verdes, 123+).
- Build frontend: npm run build --workspace @cartel/frontend.
- Migraciones puntuales: node --env-file=.env scripts/<migracion>.mjs (y .env.production para prod).
- Aplicar schema/seed: scripts del backend con db.executeMultiple.

Flujo de trabajo recomendado

- Rama desde main para cada ticket.
- Cambios aditivos; no romper auth, layout ni tests.
- Tests backend verdes + build frontend antes de mergear.
- Migración (si aplica) en local y luego en producción.
- Merge a main -> auto-deploy. Un commit por ticket.

Convenciones

- UI en español de Chile. Marca: rojo #dc2626 / #c8102e, dorado #f5a623.
- Nunca hardcodear credenciales/secretos en archivos versionados.
- SQLite: los CHECK no se pueden ALTERar; agregar columnas con ALTER ADD COLUMN; cambiar un CHECK requiere reconstruir la tabla (ver scripts de migración existentes).
- Navegación por estado en App.jsx (sin react-router); módulos declarados en src/config/nav.js con su permiso.

Funcionalidades recientes (changelog de la última fase)

- KDS (Kitchen Display System): pantalla de cocina con columnas PENDIENTE/EN_PREPARACION/LISTO (screens/Kds.jsx; controlador dispatch extendido).

- Conteo de turno: al abrir/cerrar caja se registran pollos al horno, pollos crudos, sacos de papa, merma y observaciones. NO altera el inventario real (registro operativo paralelo). Vista "Cuadre de turno" con análisis de descalce y alertas (reports.turnos).
- Resumen: tarjeta "Consumo del período" (pollos vendidos por receta + papas en kg) con selector Día/Semana/Mes/Año/Personalizado (reports.consumo-insumos). Gráficos de tendencia corregidos.
- Reposición con costo de compra: al reponer un insumo se exige el costo unitario; ingredients.cost_unit se actualiza por promedio ponderado y cada compra queda en historial. Vista "Precios de compra" con variación de precio por insumo (reports.precios-insumos).
- Cartelería pública (TV): logo/QR/contacto más grandes, fotos reales por categoría, imágenes coherentes por producto en el POS.

Dónde mirar primero (orientación de código)

- Backend rutas: apps/backend/src/index.js (mapa de endpoints).
- Controladores: apps/backend/src/controllers/*.js (reports, inventory, cashRegister, dispatch, etc.).
- Esquema y migraciones: apps/backend/db/schema.sql y apps/backend/scripts/*.mjs.
- Frontend pantallas: apps/frontend/src/screens/*.jsx ; navegación en src/config/nav.js ; render en src/App.jsx.
- Permisos/roles: src/config/roles.js + matriz role_permissions (ver docs de seguridad).

El Cartel de los Pollos — POS & Gestión (MVP)

Operación delivery-only. Inventario teórico estricto vía BOM, cierre de caja ciego, offline-first y arquitectura Zero Trust. Sin ensaladas/vegetales frescos en el modelo de negocio.

Ø=ÜÜ Documentación (equipo de desarrollo)

- docs/ARQUITECTURA.md (docs/ARQUITECTURA.md) — visión general, stack, flujo de una venta, ruteo SPA, PWA, deploy.
- docs/MODULOS.md (docs/MODULOS.md) — guía funcional de cada módulo (pantalla · endpoints · permiso · tablas · reglas).
- docs/MODELO-DATOS.md (docs/MODELO-DATOS.md) — tablas, relaciones e invariantes.
- docs/API.md (docs/API.md) — referencia de endpoints agrupada por módulo.
- docs/SEGURIDAD-RBAC.md (docs/SEGURIDAD-RBAC.md) — JWT, firma HMAC, OTP, PIN admin y matriz roles×permisos.
- docs/ONBOARDING.md (docs/ONBOARDING.md) — puesta en marcha local, scripts, convenciones, tests y despliegue.
- Referencia adicional: docs/rbac-matrix.md (docs/rbac-matrix.md) · docs/audit-events.md (docs/audit-events.md).

> El resto de este README es histórico (puede tener detalles desfasados); la fuente

> actualizada es la carpeta docs/.

Stack

- Frontend: Vite + React + TailwindCSS (UI Poka-yoke: botones grandes, sin texto libre innecesario).
- Backend: Node.js + Express (export de handler listo para Serverless/PaaS).
- DB: Turso DB (libSQL / SQLite).

Estructura del monorepo

```

el-cartel-pollos/
% % package.json          # workspaces (apps/*, packages/*)
% % README.md
% % apps/
% % % backend/
% % % % package.json
% % % % db/
% % % % % schema.sql      # DDL Turso (BOM, ventas, audit append-only, cierre)
% % % % % seed.sql        # semilla SIN ensaladas
% % % % % scripts/
% % % % % % applySchema.js # aplica schema.sql / seed.sql a Turso
% % % % % % seedUsers.js   # usuarios demo + secreto OTP gerencia
% % % % % src/
% % % % % % index.js       # rutas + montaje de middlewares
% % % % % % db.js          # cliente libSQL
% % % % % % middleware/
% % % % % % % auth.js      # JWT + roles + OTP gerencia en PUT/DELETE
% % % % % % % hmac.js     # validación anti-tamper HMAC-SHA256
% % % % % % % controllers/
% % % % % % % % auth.js    # login: emite JWT + clave de sesión HMAC
% % % % % % % % sales.js   # sync de ventas + catálogo POS
% % % % % % % % inventory.js # mermas obligatorias + alertas de stock
% % % % % % % % admin.js   # CRUD catálogo (PUT/DELETE -> OTP)
% % % % % % % % reports.js # reportes de turno (solo GERENCIA)
% % % % % % % % cashRegister.js # Cierre de Caja CIEGO
% % % % % % % services/
% % % % % % % % audit.js   # escritura append-only
% % % % % % % % sales.js   # registro transaccional + descuento BOM
% % % % % % % % sessionKeys.js # claves de sesión temporales (HMAC)
% % % % % frontend/      # Vite + React + Tailwind
% % % % % % index.html
% % % % % % vite.config.js # proxy /api -> backend
% % % % % % tailwind.config.js
% % % % % % src/
% % % % % % % main.jsx
% % % % % % % App.jsx      # shell + estado online/offline
% % % % % % % screens/
% % % % % % % % Login.jsx
% % % % % % % % Pos.jsx    # UI Poka-yoke (botones grandes)
% % % % % % % % Merma.jsx  # mermas + alertas de stock
% % % % % % % % Manage.jsx # reportes + precios (OTP si no es gerencia)
% % % % % % % % CashClose.jsx # cierre ciego (revela tras enviar)
% % % % % % % lib/
% % % % % % % % api.js     # cliente HTTP (JWT + header OTP)
% % % % % % % % crypto.js  # firma HMAC-SHA256 (Web Crypto API)
% % % % % % % % offlineStore.js # cola de ventas en IndexedDB + auto-sync
% % packages/
% % % shared/            # (futuro) tipos y constantes compartidas

```

Variables de entorno (backend)

```

TURSO_DATABASE_URL=libsql://...
TURSO_AUTH_TOKEN=...
JWT_SECRET=...          # HS256
PORT=3000

```

Pilares de seguridad (Zero Trust)

- JWT obligatorio en /api/*.
- OTP de gerencia exigido a Cajero/Preparador para PUT/DELETE (x-management-otp).
- HMAC-SHA256 firma cada venta con clave de sesión temporal; el backend rechaza payloads manipulados (PAYLOAD_MANIPULADO).
- Cierre ciego: el frontend nunca recibe el teórico antes del cierre.
- audit_logs append-only reforzado por triggers (UPDATE/DELETE abortan).

Puesta en marcha

```

npm install

# Backend: configurar credenciales
cp apps/backend/.env.example apps/backend/.env # editar TURSO_* y JWT_SECRET

# Crear esquema + semilla (productos/insumos) + usuarios demo
npm -w @cartel/backend run db:seed # schema.sql + seed.sql
npm -w @cartel/backend run seed:users # imprime credenciales y OTP de gerencia

# Cargar la carta real (40 productos) + insumos y recetas con food cost
npm -w @cartel/backend run seed:carta # 40 productos en 6 categorías
npm -w @cartel/backend run seed:recetas # 24 insumos + recetas (BOM) + tabla de food cost

# Levantar
npm -w @cartel/backend run dev # API en :3000
npm -w @cartel/frontend run dev # POS en :5173 (proxy /api -> :3000)

```

Usuarios demo (tras seed:users)

Usuario	Clave	Rol
cajero1	cajero123	CAJERO
prep1	prep123	PREPARADOR
gerente	gerente123	GERENCIA

El secreto TOTP de gerencia se imprime en consola: cárgalo en una app authenticator para autorizar PUT/DELETE de cajero/preparador (header x-management-otp).

API (resumen)

Método	Ruta	Rol / Guard
POST	/api/auth/login	público
GET	/api/products	autenticado
POST	/api/sales/sync	autenticado + HMAC
GET	/api/inventory/ingredients · /alerts	autenticado
POST	/api/inventory/merma	autenticado (reason obligatorio)
GET	/api/cash-register/current	estado de caja (ciego, sin teórico)
POST	/api/cash-register/open	CAJERO / GERENCIA (fondo inicial)
POST	/api/cash-register/movement	CAJERO / GERENCIA (depósito/ingreso)
POST	/api/cash-register/close	CAJERO / GERENCIA (ciego)
GET	/api/expenses/categories	autenticado
POST	/api/expenses	autenticado (registrar gasto)
GET	/api/expenses	solo GERENCIA
PUT/DELETE	/api/products/:id · /api/ingredients/:id	OTP gerencia si no es GERENCIA
GET	/api/reports/turn-summary · /closures · /cash-flow · /pnl	permiso reports.view
POST/PUT/DELETE	/api/inventory/ingredients[...]	permiso inventory.manage (+OTP en PUT/DELETE)
POST	/api/inventory/ingredients/:id/restock	reponer stock (+gasto opcional)
POST	/api/products · PUT/DELETE :id	permiso menu.manage (+OTP en PUT/DELETE)
GET/PUT	/api/products/:id/recipe	permiso recipes.manage (decimales)
GET/PUT	/api/dispatch[:saleId/status]	permiso dispatch.manage
GET/PUT	/api/permissions · /api/permissions/me	permiso permissions.manage

GET	/api/sales/:id/receipt	datos del comprobante (imprimir/reenviar)
GET/PUT	/api/settings	datos del negocio (PUT: permiso settings.manage +OTP)

Flujo de venta / POS

El POS exige caja abierta para vender (si está cerrada, lleva a la apertura). Con la caja abierta, "Nueva venta" ofrece dos modos:

- Venta de productos: catálogo con pestañas por categoría + buscador, carrito y cobro.
 - Venta libre: ingreso por un monto (con descripción opcional), sin descontar inventario.
- Ambas firman con HMAC, reciben N° de orden y muestran el panel de comprobante.

Flujo de venta (resumen)

- Login !' backend emite JWT + clave de sesión; el frontend la importa en memoria (crypto.js).
- POS arma el pedido y firma el payload con HMAC-SHA256.
- Con red: POST /api/sales/sync !' verifyHmac valida la firma !' registro transaccional (venta + items + descuento BOM de insumos + ajuste de inventario + auditoría) atómico.
- Sin red: la venta firmada se encola en IndexedDB y se reintenta al volver online (idempotente por client_uuid).

Finanzas

- Gastos / egresos: registrar por categoría (proveedores, sueldos, arriendo/servicios, retiros de socios), método de pago, proveedor y descripción.
 - Cuadratura de caja (ciego) con fondo: se abre y cierra caja contando billetes y monedas por denominación (CLP), con total automático; el conteo se guarda y debe cuadrar con lo declarado. El teórico se calcula como fondo + ventas_efectivo " gastos_efectivo ± movimientos. Solo los gastos en efectivo afectan el cajón. El cajero cierra a ciegas y solo ve la confirmación; el resumen del turno (ventas, gastos, balance y descuadre) se revela únicamente a quien tenga permiso reports.view (gerencia).
 - Flujo de caja (/reports/cash-flow, GERENCIA): ingresos vs egresos de todo el dinero por día, con saldo acumulado y desglose de egresos por categoría.
 - Estado de Resultados / P&L (/reports/pnl, GERENCIA): ventas " **costo de insumos (BOM, costo congelado por movimiento)** = utilidad bruta; " mermas " gastos operativos = utilidad operativa; los retiros de socios se muestran aparte (no son gasto). Incluye food cost % y márgenes. El costo unitario se congela en inventory_adjustments.unit_cost al vender/mermar, para un P&L histórico correcto aunque cambien los precios de los insumos.
- > Las marcas de tiempo (sold_at, opened_at, spent_at) se guardan en ISO 8601 UTC
> para que los rangos de período sean comparables (no usar datetime('now') en columnas
> que se filtran por rango).

Permisos por módulo

La matriz role_permissions define qué puede hacer cada rol (10 módulos). Gerencia la edita en caliente (/permissions, pantalla Permisos); el middleware requirePermission gatea cada endpoint y la nav del frontend se arma según permissions/me. El OTP de gerencia ya no es global: se aplica de forma selectiva solo a operaciones sensibles (editar/eliminar carta, insumos y permisos), no a acciones operativas (despacho, recetas).

Carta, inventario y recetas

- Carta (tabla tipo Treinta): Producto · Precio (editable inline) · Costo (calculado por receta) · Ganancia + margen % · Receta (Ver/Agregar) · Eliminar. Filtro por categoría + buscador. Endpoint /api/products/catalog calcula costo/ganancia/margen por BOM. Crear/editar/eliminar platos (SKU autogenerado, baja lógica). Las recetas rebajan el inventario al vender.
- Inventario: crear/editar/eliminar insumos + reponer stock (con gasto enlazado opcional). Borrar un insumo en uso por una receta queda bloqueado.
- Recetas (BOM): constructor por producto con cantidades enteras o decimales (ej. 0,5 pollo, 0,6 kg papas); muestra costo y margen en vivo. Al vender, descuenta del

inventario en decimales.

Cientes / Domicilios

En la confirmación de venta se puede activar entrega a domicilio: teléfono (!' autocompleta nombre y dirección de clientes recurrentes), nombre, dirección y costo de envío (suma al total). El cliente se crea/actualiza solo (upsert por teléfono). El comprobante muestra cliente, dirección y envío, y WhatsApp se envía directo al teléfono del cliente. Pantalla Clientes con buscador y acceso directo a WhatsApp.

Adiciones / Modificadores

Grupos de opciones (ej. "Presa preferida", "Salsas extra") con mín/máx y obligatoriedad, opciones con recargo opcional, asignables a productos. En el POS, tocar un producto con modificadores abre un modal para elegir; el recargo lo aplica el servidor (anti-tamper) y los modificadores aparecen en el carrito, el comprobante y el ticket de cocina.

Comprobantes e impresión

Al cobrar, el POS muestra el N° de orden y permite imprimir ticket de cocina (sin precios, ítems grandes), imprimir boleta para el cliente y enviar el comprobante por WhatsApp (link wa.me con el resumen). La impresión es vía diálogo del SO (sirve con impresora térmica 58/80mm instalada como impresora del sistema); el ancho de papel se configura en Ajustes, junto con nombre, dirección, teléfono, RUT y mensaje de pie. Los pedidos se pueden reimprimir / reenviar desde el tablero de Despacho.

Despacho (número de orden)

Cada venta recibe un número de orden correlativo por día (zona America/Santiago), asignado por el servidor al sincronizar (sin choques entre cajas ni offline; una venta offline recibe su número al reconectar). El tablero de despacho lista los pedidos del día y permite avanzarlos: Pendiente !' En preparación !' Listo !' Entregado.

PWA / Offline

La app es una PWA (vite-plugin-pwa + Workbox). El service worker precachea el app shell, por lo que el POS abre sin conexión desde cero; el catálogo usa NetworkFirst (cae a caché si no hay red). El SW solo se activa en build de producción:

```
npm -w @cartel/frontend run build
npm -w @cartel/frontend run preview # http://localhost:4173
```

Para probar offline: abrir el preview, DevTools !' Application !' Service Workers !' marcar Offline, y recargar. La app carga; las ventas quedan en cola y se sincronizan al reconectar.

Arquitectura — El Cartel de los Pollos

POS + gestión para una rotisería: venta (offline-first), caja ciega, inventario por receta (BOM), finanzas (P&L, flujo, banco), predicción de horno, carta pública y cartelera para TV. Producción: <https://cartel-pollos.onrender.com>

Stack

- Monorepo npm workspaces (apps/*).
- Backend (apps/backend): Node.js + Express (ESM) + Turso/libSQL (@libsql/client).
- Frontend (apps/frontend): Vite + React (JSX) + TailwindCSS, PWA (vite-plugin-pwa, cola offline en IndexedDB).
- Un solo servicio: Express expone la API en /api y sirve la PWA (build de Vite en apps/frontend/dist) en el mismo dominio !' sin CORS.
- Deploy: Render (render.yaml, servicio Node persistente) + Turso. Detalle en DEPLOY.md.

> ¿Por qué servidor persistente y no serverless? Las claves HMAC de sesión que
> firman las ventas viven en memoria del proceso; serverless las perdería.

Layout del repo

```
apps/backend/  
  src/index.js      # montaje de TODAS las rutas /api + estáticos del frontend  
  src/db.js         # cliente libSQL (TURSO_DATABASE_URL/_AUTH_TOKEN)  
  src/controllers/* # auth, sales, cashRegister, dispatch, inventory, admin,  
                  # recipes, modifiers, clients, expenses, bank, reports,  
                  # settings, users, permissions, audit, publicCatalog  
  src/middleware/   # auth.js (requireAuth, requireOtpForMutation), permissions.js  
                  # (requirePermission), hmac.js (verifyHmac), rateLimit.js  
  src/services/     # permissions.js (matriz RBAC), audit.js (writeAudit)  
  src/config/roles.js # catálogo de roles  
  db/schema.sql     # esquema completo + triggers (append-only de auditoría)  
  db/seed.sql       # datos base idempotentes  
  scripts/          # provision, migrate-*, set-admin-pin, backup, e2e, etc.  
  test/            # vitest (helpers, suites por dominio)  
apps/frontend/  
  src/main.jsx      # entrypoint: enruta páginas PÚBLICAS por pathname  
  src/App.jsx       # shell autenticado (estado `screen`, sin react-router)  
  src/screens/*.jsx # una pantalla por módulo (ver MODULOS.md)  
  src/components/   # PeriodNav, ui/States, AbrirCajaModal, denoms, etc.  
  src/config/nav.js  # arquitectura de información (módulos x permiso) – fuente única  
  src/config/roles.js # etiquetas de rol para la UI  
  src/lib/          # api.js, offlineStore.js, receipt.js, crypto.js, print.js,  
                  # productName.js, categoryAssets.js  
  public/           # landing.html, logo.jpeg, hero-*.jpg, iconos PWA  
  docs/             # esta documentación  
  render.yaml       # blueprint de Render
```

Ruteo (sin react-router)

- App autenticada: App.jsx mantiene un estado screen; el menú lateral se arma desde config/nav.js filtrando por permiso. Cada pantalla se renderiza con {screen === 'x' && <Pantalla/>} y hay un guard por permiso (<Forbidden/> si falta el permiso del ítem). Incluye auto-logout por inactividad (30 min) y manejo global del evento session-expired (401) para cerrar sesión limpio.
- Páginas públicas (sin login), resueltas en main.jsx por window.location.pathname:
 - /catalogo/:slug | /menu/:slug | /carta/:slug !' PublicCatalog (carta para clientes, pedido por WhatsApp).
 - /cartelera/:slug | /tv/:slug !' PublicCartelera (menuboard 16:9 para TV).
 - /landing.html !' landing estática (servida desde public/).

Flujo de una venta (offline-first + anti-tamper)

- Al login el backend entrega { token, user, session:{ id, key } }. La key (clave HMAC de la sesión) se guarda solo en memoria del cliente (lib/crypto.js).
- El POS arma el payload de la venta y lo firma con HMAC-SHA256 usando esa clave.
- POST /api/sales/sync pasa por verifyHmac (middleware) !' si la firma no cuadra con la sesión, se rechaza (anti-tamper) y se audita.

- controllers/sales.syncSale persiste sales + sale_items, **descuenta inventario por BOM (product_recipes != inventory_adjustments tipo VENTA con costo congelado**), asigna order_number correlativo por business_day y escribe audit_logs.
- Offline: si no hay red, lib/offlineStore.js encola la venta en IndexedDB; flushQueue() la sincroniza al recuperar conexión (idempotente por client_uuid).

Datos y seguridad (resumen)

- Auth: JWT HS256; requireAuth protege todo /api (salvo /api/auth/login y /api/public/...). Ver SEGURIDAD-RBAC.md.
- RBAC: matriz role_permissions (data-driven) + requirePermission(perm).
- OTP (TOTP): requireOtpForMutation en mutaciones sensibles (editar carta, ajustes, permisos). PIN de admin (bcrypt) para ajuste de stock auditado.
- Auditoría: audit_logs append-only (triggers bloquean UPDATE/DELETE).
- Modelo de datos completo en MODELO-DATOS.md.

PWA

- registerType: 'autoUpdate'; navigateFallback: '/index.html' con denylist [/^Vapi\$/, /^Vlanding/]. El SW precachea la app; tras un deploy, una pantalla ya abierta necesita una recarga para tomar la versión nueva (ver nota en DEPLOY.md).

Deploy y variables de entorno

- Render (render.yaml): buildCommand: npm ci --include=dev && npm run build, startCommand: npm start, healthCheckPath: /health, autoDeploy: true.
- Variables:

Variable	Uso
TURSO_DATABASE_URL	URL libSQL de la base (secreto)
TURSO_AUTH_TOKEN	token de Turso (secreto)
JWT_SECRET	firma de tokens (Render lo genera)
JWT_TTL	vigencia del token (def. 12h)
NODE_ENV	production ("` serverless para que el server haga listen)
PORT	puerto local (def. 3000)

- Bootstrap de una base nueva: scripts/provision.mjs (aplica schema.sql + seed.sql y crea un usuario GERENCIA con contraseña fuerte + OTP). Ver ONBOARDING.md.

Tests

- Vitest (apps/backend/test), single-fork (BD en memoria + Map HMAC compartidos).
npm test --workspace @cartel/backend.

Guía de módulos

Cada módulo del menú lateral (apps/frontend/src/config/nav.js) tiene una pantalla en apps/frontend/src/screens/. Formato por módulo: ****Qué hace · Pantalla · Endpoints · Permiso · Tablas · Reglas clave****. Para los endpoints ver API.md; para permisos ver SEGURIDAD-RBAC.md; para tablas ver MODELO-DATOS.md.

OPERACIÓN

Ø=ÞÒ Vender (pos)

- Qué hace: punto de venta. Venta de productos (con modificadores) o venta libre (monto).
- Pantalla: Pos.jsx · Permiso: pos.sell
- Endpoints: GET /products, POST /sales/sync (firma HMAC), GET /cash-register/current, GET/POST /clients, GET /products/:id/modifiers, GET /settings.
- Tablas: sales, sale_items, products, clients, inventory_adjustments.
- Reglas: exige caja abierta (modal de apertura obligatorio si está cerrada); la venta se firma con HMAC (anti-tamper) y descuenta inventario por BOM; offline-first (cola IndexedDB, idempotente por client_uuid).

Ø>ÝÞ Ventas (ventas)

- Qué hace: historial de transacciones, ver/reimprimir comprobante, anular.
- Pantalla: Ventas.jsx · Permiso: pos.sell (anular: sales.void)
- Endpoints: GET /sales, GET /sales/:id/receipt, POST /sales/:id/void.
- Tablas: sales, sale_items.
- Reglas: anular cambia status a ANULADA (audita); no borra.

Ø=ÝS Venta retroactiva (retroactiva)

- Qué hace: registrar una venta con fecha/hora pasada.
- Pantalla: VentaRetroactiva.jsx · Permiso: sales.backdate
- Endpoints: POST /sales/backdate.
- Tablas: sales (is_backdated, backdate_reason).
- Reglas: flujo separado de gerencia, máx. ~30 días atrás, justificación obligatoria, fuertemente auditada.

Ø=Üµ Caja (cash)

- Qué hace: apertura con fondo, depósitos/ingresos y cierre ciego con arqueo.
- Pantalla: CashClose.jsx (usa AbrirCajaModal, denoms) · Permiso: cash.operate
- Endpoints: GET /cash-register/current, POST /cash-register/open|movement|close.
- Tablas: cash_sessions, cash_movements, cash_register_closures.
- Reglas: el operador declara efectivo/POS/transferencias sin ver el teórico; el backend calcula el teórico y la diferencia/descuadre (se persisten). Solo una sesión OPEN a la vez (índice único). El resumen del turno solo lo ve quien tenga reports.view.

Ø=Þõ Despacho (despacho)

- Qué hace: tablero de pedidos con número de orden y estados.
- Pantalla: Despacho.jsx · Permiso: dispatch.manage
- Endpoints: GET /dispatch, PUT /dispatch/:saleId/status.
- Tablas: sales (dispatch_status, order_number, business_day).
- Reglas: estados PENDIENTE !' EN_PREPARACION !' LISTO !' ENTREGADO.

Ø=Ý. Predicción de horno (prediccion)

- Qué hace: recomienda cuántos pollos hornear por día/franja.
- Pantalla: Prediccion.jsx · Permiso: forecast.view
- Endpoints: GET /reports/forecast.
- Tablas: sales, sale_items.
- Reglas: demanda en pollo-equivalente por día de semana con recencia (semanas recientes pesan más), ajuste por clima (open-meteo) y feriados chilenos; plan de horneadas por ventana de servicio.

Ø=ÝÑÞ Mermas (merma)

- Qué hace: registrar pérdida de insumos.
- Pantalla: Merma.jsx · Permiso: inventory.merma
- Endpoints: POST /inventory/merma, GET /inventory/ingredients.
- Tablas: inventory_adjustments (tipo MERMA), ingredients.
- Reglas: motivo obligatorio; descuento stock; alimenta la línea "mermas" del P&L.

CATÁLOGO

Ø<BW Carta (carta)

- Qué hace: ABM de productos, precios, recetas (BOM), foto, visibilidad y catálogo virtual (link + QR).
- Pantalla: Carta.jsx · Permiso: menu.manage (recetas: recipes.manage)
- Endpoints: GET /products/catalog, POST /products, PUT/DELETE /products/:id (OTP), GET/PUT /products/:id/recipe, GET/PUT /settings.
- Tablas: products, product_recipes, ingredients, business_settings.
- Reglas: validación de nombre (descriptivo, sin códigos tipo UPBEB125); costo/ganancia/margen calculados por BOM; in_catalog controla visibilidad pública; editar precio/nombre exige OTP a roles no gerenciales.

Ø=ÜË Cartelera (cartelera)

- Qué hace: arma una cartelera de precios imprimible desde la carta actual.
- Pantalla: Cartelera.jsx · Permiso: menu.manage
- Endpoints: GET /products/catalog.
- Reglas: selección de productos + filtro por categoría; relacionada con la cartelera pública para TV (/cartelera/:slug).

'(Modificadores (modificadores)

- Qué hace: grupos de adiciones/opciones (presa, salsas) y su enlace a productos.
- Pantalla: Modificadores.jsx · Permiso: menu.manage
- Endpoints: GET /modifiers, POST/DELETE /modifiers/groups, PUT /modifiers/groups/:id/products, POST/DELETE /modifiers/options.
- Tablas: modifier_groups, modifier_options, product_modifier_groups.
- Reglas: cada grupo define min/max_select y si es requerido; cada opción tiene price_delta.

Ø=Üæ Inventario (inventario)

- Qué hace: insumos (stock teórico), alertas de stock bajo, reposición y ajuste auditado con PIN.
- Pantalla: Inventario.jsx · Permiso: inventory.manage
- Endpoints: GET /inventory/ingredients|alerts, POST /inventory/ingredients, PUT/DELETE /inventory/ingredients/:id (OTP), POST /inventory/ingredients/:id/restock, POST /inventory/ingredients/:id/set-stock (PIN admin + rate-limit).
- Tablas: ingredients, inventory_adjustments.
- Reglas: stock teórico que descuenta el BOM al vender; el set-stock (reemplaza/suma) exige el PIN de administrador (bcrypt) y queda auditado.

FINANZAS (permiso base reports.view)

Ø=ÜË Resumen (resumen)

- KPIs del día/turno. Pantalla: Resumen.jsx · Endpoints: GET /reports/dashboard, GET /reports/turn-summary.

Ø=Ü± Movimientos (movimientos)

- Ledger unificado (ventas, gastos, movimientos de caja). Pantalla: Movimientos.jsx · Endpoints: GET /reports/movements.

Ø=Ü, Gastos (gastos)

- Qué hace: registrar egresos y retiros. Pantalla: Gastos.jsx · Permiso: expenses.manage (listar: reports.view).
- Endpoints: GET /expenses, GET /expenses/categories, POST /expenses.
- Tablas: expenses, expense_categories (kind OPERATIVO/RETIRO).

- Reglas: el método de pago afecta flujo y, si es efectivo en sesión, la cuadratura.

Ø=ÜÈ Flujo de caja (flujo)

- Ingresos/egresos por día + saldo acumulado. Pantalla: Flujo.jsx · Endpoints: GET /reports/cash-flow.

Ø<ßæ Banco (banco)

- Qué hace: movimientos bancarios + conciliación; alimenta la "realidad bancaria" del P&L.
- Pantalla: Banco.jsx · Endpoints: GET /bank/summary|movements|reconcile, POST /bank/movements (expenses.manage), PUT /bank/movements/:id/reconcile (expenses.manage).
- Tablas: bank_movements, business_settings (bank_balance).

Ø>Ýî P&L (pnl)

- Estado de resultados. Pantalla: Pnl.jsx · Endpoints: GET /reports/pnl.
- Fórmula: Utilidad bruta = Ventas " Costo insumos (BOM congelado, vía inventory_adjustments); Utilidad operativa = bruta " Mermas " Gastos operativos; Retiros aparte; sección "realidad bancaria" (egresos reales del banco).

Ø=ÜÊ Estadísticas (estadísticas)

- Ventas por día/método, ranking de productos, comparativo de período. Pantalla: Estadisticas.jsx · Endpoints: GET /reports/stats.

CLIENTES

Ø=Üe Clientes (clientes)

- ABM de clientes para domicilios (lookup por teléfono). Pantalla: Clientes.jsx · Permiso: pos.sell · Endpoints: GET/POST /clients · Tablas: clients.

ADMINISTRACIÓN

Ø<ßê Negocio (ajustes)

- Datos del local: nombre, dirección, WhatsApp, slug del catálogo, formas de entrega, ancho de papel, PIN de admin.
- Pantalla: Ajustes.jsx · Permiso: settings.manage · Endpoints: GET/PUT /settings (OTP), PUT /settings/admin-pin (OTP) · Tablas: business_settings.

Ø=Üd Usuarios (usuarios)

- Crear/editar staff, resetear contraseña, activar/desactivar.
- Pantalla: Usuarios.jsx · Permiso: permissions.manage · Endpoints: GET/POST /users, PUT /users/:id, POST /users/:id/password · Tablas: users.

Ø=Ý Roles y permisos (permisos)

- Matriz rol × permiso editable.
- Pantalla: Permisos.jsx · Permiso: permissions.manage · Endpoints: GET/PUT /permissions (OTP), GET /permissions/me · Tablas: role_permissions.
- Reglas: anti-lockout — GERENCIA/ADMIN nunca pierden permissions.manage.

Ø=Þáp Auditoría (auditoría)

- Bitácora de actividad (solo lectura).
- Pantalla: Auditoria.jsx · Permiso: audit.view · Endpoints: GET /audit, GET /audit/actions · Tablas: audit_logs (append-only, severidad INFO/WARN/ALERT).

VISTAS PÚBLICAS (sin login)

Carta pública — PublicCatalog.jsx

- Rutas /carta/:slug · /catalogo/:slug · /menu/:slug. Endpoint: GET /public/catalog/:slug.
- Muestra productos con in_catalog=1; el cliente arma pedido y lo envía por WhatsApp según las formas de entrega habilitadas. Tablas: products, business_settings.

Cartelera TV — PublicCartelera.jsx

- Rutas /cartelera/:slug · /tv/:slug. Endpoint: GET /public/catalog/:slug.
- Menuboard 16:9 rotativo (slides hero + columnas), QR de WhatsApp, auto-refresco cada 60s, autoescalado a la pantalla.

Landing — public/landing.html

- Sitio estático de marca (link-in-bio) con CTAs a la carta y a WhatsApp, reseñas y redes. Servido en / landing.html.

Modelo de datos

Fuente: apps/backend/db/schema.sql (Turso/libSQL = SQLite). IDs TEXT (UUID), timestamps TEXT (ISO/datetime('now')). PRAGMA foreign_keys = ON.

Tablas

Tabla	Propósito	Columnas clave	Relaciones
users	Operadores y gerencia	username(unique), password_hash, role, otp_secret, is_active	referida por casi todas las tablas con user_id
role_permissions	Matriz RBAC	PK(role,permission), allowed	— (roles validados en código)
business_settings	Datos del local (fila única id=1)	name, address, whatsapp, catalog_slug, paper_width, admin_pin_hash, bank_balance, pickup/delivery_enabled	—
ingredients	Insumos del inventario teórico	name(unique), unit, stock_qty, min_stock_qty, cost_unit	—
products	Productos de venta	sku(unique), name, price, category, in_catalog, is_active	—
product_recipes	BOM: insumo por unidad de producto	qty_per_unit, unique(product_id,ingredient_id)	product_id!'products(CASCADE), ingredient_id!'ingredients(RESTRICT)
sales	Cabecera de venta	client_uuid(unique), total, payment_method, status, payload_hash, business_day, order_number, dispatch_status, is_backdated, sold_at	user_id!'users; client_id!'clients (lógico)
sale_items	Detalle de venta (precio congelado)	qty, unit_price, modifiers(JSON), line_total	sale_id!'sales(CASCADE), product_id!'products(RESTRICT)
clients	Clientes para domicilio	phone(unique), name, address	referida por sales.client_id
modifier_groups	Grupos de adiciones	min_select, max_select, is_required	—
modifier_options	Opciones del grupo	name, price_delta	group_id!'modifier_groups(CASCADE)
product_modifier_groups	Enlace producto!"grupo	PK(product_id,group_id)	!'products, !'modifier_groups (CASCADE)
inventory_adjustments	Movimientos de stock (COGS)	type(MERMA/VENTA/REPOSICION/CONTEO), qty_delta, unit_cost(congelado), reason	ingredient_id!'ingredients, user_id!'users, sale_id!'sales(SET NULL)
cash_sessions	Apertura/cierre de caja con fondo	opening_float, status(OPEN/CLOSED), closure_id	opened_by!'users
cash_movements	Efectivo no-venta de la sesión	type(DEPOSITO/INGRESO), amount, reason	session_id!'cash_sessions(CASCADE), user_id!'users
cash_register_closures	Cierre ciego (declarado vs teórico)	efectivo/pos/transferencias_declarado, _teorico, diff_, has_descuadre	user_id!'users, session_id!'cash_sessions(SET NULL)
expense_categories	Categorías de gasto	name(unique), kind(OPERATIVO/RETIRO)	referida por expenses.category_id
expenses	Egresos/gastos	amount, payment_method, supplier, description, spent_at	category_id!'expense_categories, user_id!'users

bank_movements	Conciliación bancaria	fecha, amount, direction(INGRESO/EGRESO), reconciled, category	created_by!'users; sale_id (lógico)
audit_logs	Bitácora append-only	action, entity, severity(INFO/WARN/ALERT), metadata(JSON), ip_address	user_id!'users (nullable)

Relaciones (vista textual)

```
users % % < sales % % < sale_items >% % products % % < product_recipes >% % ingredients
                                     %                                     %^2
                                     % % modifiers(JSON)                % (descuento BOM)
products % % < product_modifier_groups >% % modifier_groups % % < modifier_options
sales % % < inventory_adjustments >% % ingredients                (type=VENTA: COGS congelado)
users % % < cash_sessions % % < cash_movements
          % % < cash_register_closures
expense_categories % % < expenses % % users
bank_movements % % users
clients % % < sales
```

Notas e invariantes

- Append-only de auditoría: triggers audit_logs_no_update / audit_logs_no_delete hacen RAISE(ABORT, ...) ante cualquier UPDATE/DELETE sobre audit_logs.
- Una sola caja abierta: índice único parcial uniq_one_open_session sobre cash_sessions(status) WHERE status='OPEN'.
- Conciliación sin duplicados: índice único uniq_bankmov sobre bank_movements(fecha, amount, description, bank_type, direction).
- COGS / food cost: el costo de insumos del P&L sale de inventory_adjustments tipo VENTA (SUM(ABS(qty_delta)*unit_cost)), con costo congelado al momento de la venta — no se recalcula con el costo actual del insumo.
- sales: business_day (zona America/Santiago) + order_number correlativo por día (despacho); payload_hash guarda la firma HMAC; client_uuid da idempotencia offline.
- Aplicar el esquema: usar db.executeMultiple(sql) (parseo server-side, soporta triggers) — no dividir por ; en cliente contra Turso remoto.

Referencia de API

Base: /api. Derivada de apps/backend/src/index.js. Todo /api exige JWT (requireAuth) salvo /api/auth/login y /api/public/*. Middlewares adicionales: requirePermission(perm), requireOtpForMutation (OTP de gerencia), verifyHmac (firma de venta), rateLimit.
Auth: header Authorization: Bearer <jwt>. OTP: header x-management-otp: <código>.

Público (sin JWT)

Método	Ruta	Notas
POST	/api/auth/login	rate-limit 30/5min. Devuelve {token, user, session:{id,key}}
GET	/api/public/catalog/:slug	catálogo de vitrina (carta pública y cartelera)
GET	/health	healthcheck

Sesión / permisos

Método	Ruta	Permiso	Propósito
GET	/api/permissions/me	(autenticado)	permisos efectivos del usuario (la UI muestra/oculta)
GET	/api/products	(autenticado)	catálogo para el POS

Ventas

Método	Ruta	Permiso / mw	Propósito
POST	/api/sales/sync	pos.sell + verifyHmac	sincroniza una venta firmada (anti-tamper)
POST	/api/sales/backdate	sales.backdate	venta con fecha pasada (auditada)
GET	/api/sales	pos.sell	listado de transacciones
GET	/api/sales/:id/receipt	(autenticado)	comprobante
POST	/api/sales/:id/void	sales.void	anula una venta

Caja

Método	Ruta	Permiso	Propósito
GET	/api/cash-register/current	(autenticado)	estado de la caja (ciego)
POST	/api/cash-register/open	cash.operate	abrir con fondo
POST	/api/cash-register/movement	cash.operate	depósito/ingreso
POST	/api/cash-register/close	cash.operate	cierre ciego

Despacho

Método	Ruta	Permiso
GET	/api/dispatch	dispatch.manage
PUT	/api/dispatch/:saleId/status	dispatch.manage

Inventario

Método	Ruta	Permiso / mw
GET	/api/inventory/ingredients	(autenticado)
GET	/api/inventory/alerts	(autenticado)
POST	/api/inventory/merma	inventory.merma
POST	/api/inventory/ingredients	inventory.manage
PUT	/api/inventory/ingredients/:id	inventory.manage + OTP
DELETE	/api/inventory/ingredients/:id	inventory.manage + OTP
POST	/api/inventory/ingredients/:id/restock	inventory.manage
POST	/api/inventory/ingredients/:id/set-stock	inventory.manage + rate-limit (valida PIN admin)

Carta / recetas / modificadores

Método	Ruta	Permiso / mw
GET	/api/products/catalog	menu.manage
POST	/api/products	menu.manage
PUT/DELETE	/api/products/:id	menu.manage + OTP
GET/PUT	/api/products/:id/recipe	recipes.manage
GET	/api/products/:id/modifiers	(autenticado)
GET	/api/modifiers	menu.manage
POST/DELETE	/api/modifiers/groups[:id]	menu.manage
PUT	/api/modifiers/groups/:id/products	menu.manage
POST/DELETE	/api/modifiers/options[:id]	menu.manage

Gastos / clientes

Método	Ruta	Permiso
GET	/api/expenses/categories	(autenticado)
GET	/api/expenses	reports.view
POST	/api/expenses	expenses.manage
GET	/api/clients	(autenticado)
POST	/api/clients	pos.sell

Reportes (todos reports.view, sal

Método	Ruta	Permiso
GET	/api/reports/turn-summary	reports.view
GET	/api/reports/closures	reports.view
GET	/api/reports/cash-flow	reports.view
GET	/api/reports/pnl	reports.view
GET	/api/reports/stats	reports.view

GET	/api/reports/dashboard	reports.view
GET	/api/reports/movements	reports.view
GET	/api/reports/export	reports.view (CSV)
GET	/api/reports/forecast	forecast.view

Banco

Método	Ruta	Permiso
GET	/api/bank/summary · /api/bank/movements · /api/bank/reconcile	reports.view
POST	/api/bank/movements	expenses.manage
PUT	/api/bank/movements/:id/reconcile	expenses.manage

Negocio / usuarios / permisos / a

Método	Ruta	Permiso / mw
GET	/api/settings	(autenticado)
PUT	/api/settings	settings.manage + OTP
PUT	/api/settings/admin-pin	settings.manage + OTP
GET/POST	/api/users	permissions.manage
PUT	/api/users/:id	permissions.manage
POST	/api/users/:id/password	permissions.manage
GET	/api/permissions	permissions.manage
PUT	/api/permissions	permissions.manage + OTP
GET	/api/audit · /api/audit/actions	audit.view

Errores

Formato { "error": "CODIGO" } con HTTP apropiado (401 TOKEN_AUSENTE/INVALIDO, 403 PERMISO_DENEGADO, 409 CAJA_YA_ABIERTA, etc.). El cliente mapea códigos a mensajes en src/components/ui/States.jsx (humanizeError).

Seguridad y RBAC

Autenticación (JWT)

- Login POST /api/auth/login (rate-limit 30/5min) !' { token, user, session:{id,key} }.
- JWT HS256 firmado con JWT_SECRET (TTL JWT_TTL, def. 12h). El payload lleva sub(userId), role, username.
- requireAuth (src/middleware/auth.js) valida el token en todo /api (excepto /api/auth/login y /api/public/*). Sin token !' 401 TOKEN_AUSENTE; inválido !' 401 TOKEN_INVALIDO.
- El frontend guarda el JWT en localStorage; ante 401 de sesión dispara el evento session-expired y cierra sesión. Auto-logout por inactividad (30 min).

Firma HMAC de ventas (anti-tamper)

- En el login el servidor genera una clave de sesión HMAC (session.key) y la guarda en un Map en memoria del proceso (por eso el deploy es un servidor persistente, no serverless).
- El cliente guarda la key solo en memoria (lib/crypto.js) y firma cada venta (HMAC-SHA256 sobre el payload canónico).
- POST /api/sales/sync pasa por verifyHmac: recomputa la firma con la clave de la sesión; si no coincide, rechaza y audita. Evita que se inyecten/alteren ventas.

OTP de gerencia (TOTP)

- requireOtpForMutation (otplib) exige un código TOTP (header x-management-otp) para mutaciones sensibles: editar carta (PUT/DELETE /products/:id), insumos (PUT/DELETE /inventory/ingredients/:id), ajustes del negocio (PUT /settings, PUT /settings/admin-pin) y permisos (PUT /permissions).
- Gerencia/Admin tienen secreto OTP (se entrega en provision.mjs). Roles operativos que intenten estas acciones requieren el OTP de gerencia.

PIN de administrador (ajuste de stock)

- business_settings.admin_pin_hash (bcrypt). Se fija con scripts/set-admin-pin.mjs <PIN> o PUT /settings/admin-pin.
- El ajuste de stock auditado (POST /inventory/ingredients/:id/set-stock) valida el PIN y aplica rate-limit anti-fuerza-bruta; queda registrado en audit_logs.

Auditoría append-only

- audit_logs es inmutable: triggers audit_logs_no_update/no_delete abortan cualquier UPDATE/DELETE. Se escribe con `services/audit.writeAudit({action, entity, entityId, severity, metadata, ip})`. Severidades: INFO/WARN/ALERT`.

RBAC (matriz rol × permiso)

- Catálogo de permisos y defaults en src/services/permissions.js; roles en src/config/roles.js. La matriz vive en role_permissions y es editable desde la UI (Roles y permisos). requirePermission(perm) la consulta (cacheada en memoria, invalidada al cambiar).
- Anti-lockout: GERENCIA y ADMIN siempre conservan permissions.manage.

Roles

Rol	Etiqueta	Resumen
CAJERO	Cajero	Vende y cobra; opera su caja
SUPERVISOR	Supervisor	Cajero + anula ventas, gastos y reportes operativos
PREPARADOR	Cocina	Producción: despacho, predicción, mermas, inventario, recetas

DESPACHO	Despacho	Tablero de despacho y entrega
GERENCIA	Gerencia	Dueño/a: todo el negocio (superadmin)
ADMIN	Administrador	Todos los permisos, incl. matriz (superadmin)

Matriz de permisos por defecto

' = permitido por defecto. Editable por rol salvo el candado anti-lockout.

Permiso	CAJERO	SUPERVISOR	PREPARADOR	DESPACHO	GERENCIA	ADMIN
pos.sell (vender)	'	'	.	.	'	'
sales.void (anular)	.	'	.	.	'	'
sales.backdate (retroactiva)	.	.	.	.	'	'
cash.operate (caja)	'	'	.	.	'	'
dispatch.manage (despacho)	'	'	'	'	'	'
forecast.view (predicción)	'	'	'	'	'	'
expenses.manage (gastos)	.	'	.	.	'	'
inventory.merma (mermas)	'	'	'	.	'	'
inventory.manage (insumos)	.	.	'	.	'	'
recipes.manage (recetas)	.	.	'	.	'	'
menu.manage (carta)	.	.	.	.	'	'
reports.view (reportes/P&L)	.	'	.	.	'	'
settings.manage (negocio)	.	.	.	.	'	'
audit.view (auditoría)	.	.	.	.	'	'
permissions.manage (permisos)	.	.	.	.	' Ø=Ý	' Ø=Ý

Ø=Ý = anti-lockout (no se puede quitar a GERENCIA/ADMIN).

Otras protecciones

- Cabeceras: X-Content-Type-Options, X-Frame-Options: SAMEORIGIN, Referrer-Policy.
- trust proxy = 1 (IP real tras el proxy de Render, para auditoría y rate-limit).
- Body JSON limitado a 256kb.
- Nunca commitar secretos: JWT_SECRET/TURSO_* van por variables de entorno.

Matriz de Roles y Permisos (RBAC)

Fuente de verdad en código:

- Roles: apps/backend/src/config/roles.js (+ espejo apps/frontend/src/config/roles.js)
- Permisos y defaults: apps/backend/src/services/permissions.js
- Navegación por permiso: apps/frontend/src/config/nav.js
- Persistencia: tabla role_permissions (rol × permiso). Editable en vivo desde Administración !' Roles y permisos.

> El acceso NO se decide por rol hardcodedo, sino por la matriz role_permissions. Los roles son extensibles: agregar uno se hace en el catálogo + defaults (ver implementation-summary).

Roles

Rol	Tipo	Descripción
Cajero (CAJERO)	Operación	Vende y cobra. Opera su caja.
Supervisor (SUPERVISOR)	Operación	Cajero + aprueba anulaciones/ descuentos y ve reportes operativos.
Cocina (PREPARADOR)	Operación	Producción: despacho, predicción de horno, mermas, inventario, recetas.
Despacho (DESPACHO)	Operación	Tablero de despacho y entregas.
Gerencia (GERENCIA)	Administración	Dueño/a: finanzas, catálogo, usuarios, configuración.
Administrador (ADMIN)	Administración	Todo + matriz de permisos + auditoría (rol de sistema).

GERENCIA y ADMIN reciben secreto OTP (TOTP) al crearse y omiten el OTP en mutaciones sensibles (lo aprueban directamente). Son superadmins: nunca pierden permissions.manage (anti-lockout).

Catálogo de permisos

Permiso	Acción	Grupo
pos.sell	Vender en POS	Operación
sales.void	Anular ventas (desacoplado de reportes)	Operación
cash.operate	Abrir/cerrar caja, movimientos de efectivo	Operación
dispatch.manage	Tablero de despacho	Operación
forecast.view	Ver predicción de horno	Operación
expenses.manage	Registrar gastos	Operación
inventory.merma	Registrar mermas	Inventario
inventory.manage	Gestionar insumos (CRUD, reposición, ajuste con PIN)	Inventario
recipes.manage	Gestionar recetas (BOM)	Catálogo
menu.manage	Gestionar carta y modificadores	Catálogo
reports.view	Ver reportes, P&L, flujo, banco, estadísticas, movimientos	Finanzas
settings.manage	Editar datos del negocio, PIN admin	Administración
audit.view	Ver auditoría/actividad	Administración

permissions.manage	Administrar usuarios y la matriz de permisos	Administración
--------------------	--	----------------

Matriz por defecto (least-privilege)

' = permitido por defecto · vacío = denegado · Ø=Ý = forzado siempre (anti-lockout).

Permiso	Cajero	Supervisor	Cocina	Despacho	Gerencia	Admin
pos.sell	'	'			'	'
sales.void		'			'	'
cash.operate	'	'			'	'
dispatch.manage	'	'	'	'	'	'
forecast.view	'	'	'	'	'	'
expenses.manage		'			'	'
inventory.merma	'	'	'		'	'
inventory.manage			'		'	'
recipes.manage			'		'	'
menu.manage					'	'
reports.view		'			'	'
settings.manage					'	'
audit.view					'	'
permissions.manage					Ø=Ý	Ø=Ý

> Los defaults se siembran al inicializar la matriz y se completan con scripts/migrate-perms.mjs. Cualquier celda es editable en vivo (excepto permissions.manage de los superadmins).

Restricciones y notas operativas

- Anular ventas ahora requiere sales.void (antes reports.view). Un rol que solo ve reportes ya no puede anular. Recomendado: dar sales.void a Supervisor/Gerencia.
- Ajuste manual de stock requiere inventory.manage + PIN de administrador (doble control; queda en auditoría con stock anterior/nuevo/motivo).
- Mutaciones sensibles (PUT/DELETE de catálogo, insumos, permisos, settings) exigen OTP de gerencia/admin salvo que el actor sea GERENCIA/ADMIN.
- Despacho es el rol más acotado: solo ve el tablero y la predicción.
- Cocina no vende ni cobra; gestiona producción e inventario.
- El menú se filtra por permiso y además hay guard por pantalla (defensa en profundidad): aunque alguien fuerce la ruta, ve "Acceso restringido".

Aprobaciones (hooks preparados)

- Anulación / descuento sobre umbral !' se recomienda exigir aprobación de Supervisor. Hoy sales.void ya separa la capacidad; el umbral con aprobación queda como hook documentado (ver implementation-summary !' "Aprobación de supervisor").

Eventos de Auditoría y Trazabilidad

La app registra eventos sensibles en la tabla audit_logs, que es inmutable:
triggers SQL (audit_logs_no_update, audit_logs_no_delete) bloquean UPDATE y DELETE.
Se escribe con el servicio central apps/backend/src/services/audit.js (writeAudit).
Se consulta (solo lectura) en Administración !' Auditoría (permiso audit.view).

Esquema del evento

```
// audit_logs
{
  id: string,           // uuid
  user_id: string|null, // autor; null = evento de sistema
  action: string,       // ver catálogo abajo
  entity: string,       // tabla/recurso afectado (sales, ingredients, users, ...)
  entity_id: string|null,
  severity: 'INFO' | 'WARN' | 'ALERT',
  metadata: string|null, // JSON serializado con el detalle del evento
  ip_address: string|null,
  created_at: string    // datetime('now') UTC
}
```

Helper de escritura:

```
import { writeAudit } from '../services/audit.js';
await writeAudit({ userId, action, entity, entityId, severity, ip, metadata: { ... } });
```

Catálogo de acciones registradas (estado actual)

Acción	Severidad	Entidad	Metadata clave
LOGIN_OK	INFO	users	—
LOGIN_FAIL	WARN	users	{ username }
SALE_SYNC	INFO	sales	{ total, payment_method, offline }
SALE_FREE	INFO	sales	{ free_amount, payment_method, note }
SALE_VOID	ALERT	sales	{ order_number, total, reason }
CASH_OPEN	INFO	cash_sessions	{ opening_float }
CASH_DEPOSITO / CASH_INGRESO	INFO	cash_movements	{ amount, reason }
CASH_CLOSE	INFO/ALERT	cash_register_closures	{ diff_total, has_descuadre, fondo }
STOCK_AJUSTE	WARN	ingredients	{ ingredient, stock_anterior, stock_nuevo, delta, motivo }
STOCK_PIN_REJECT	ALERT	ingredients	—
INV_MERMA / INV_REPOSICION	WARN/INFO	inventory_adjustments	{ ingredient, delta, reason }
PRODUCT_CREATE/ UPDATE/DELETE	INFO/WARN	products	{ before, after }
INGREDIENT_CREATE/ UPDATE/DELETE	INFO/WARN	ingredients	—
RECIPE_UPDATE	INFO	product_recipes	{ lineas }
EXPENSE_CREATE	INFO	expenses	{ amount, payment_method, category }
BANK_MOVEMENT	INFO	bank_movements	{ amount, direction }

DISPATCH_STATUS	INFO	sales	{ order_number, status }
USER_CREATE	INFO	users	{ username, role }
USER_UPDATE	INFO	users	{ full_name, role, is_active }
USER_PASSWORD_RESET	WARN	users	—
PERMISSION_UPDATE	INFO	role_permissions	{ role, permission, allowed }
SETTINGS_UPDATE	INFO	business_settings	—
ADMIN_PIN_SET	WARN	business_settings	—
OTP_MISSING / OTP_REJECT / OTP_GRANTED	WARN/ALERT/INFO	(ruta)	{ method }
HMAC_REJECT	ALERT	sales	{ sessionId }

Acciones "sensibles" (filtradas)

LOGIN_FAIL, SALE_VOID, STOCK_AJUSTE, STOCK_PIN_REJECT, HMAC_REJECT, OTP_REJECT, OTP_MISSING, PERMISSION_UPDATE, ADMIN_PIN_SET, CASH_CLOSE, USER_CREATE, USER_UPDATE, USER_PASSWORD_RESET, INV_MERMA (definidas en apps/backend/src/controllers/audit.js !' SENSITIVE).

API

- GET /api/audit?from&to&severity&action&q&sensitive=1&limit (perm audit.view) !' lista enriquecida con el usuario.
- GET /api/audit/actions !' catálogo de acciones presentes + lista sensible.

Eventos cubiertos vs. pedidos

Evento pedido	Estado
login/logout	' login (OK/FAIL). Logout es cliente (sin endpoint); ver TODO.
apertura/cierre de caja	' CASH_OPEN / CASH_CLOSE
anulación de ventas	' SALE_VOID (ALERT)
descuentos manuales	Ø=¿Á quedan en la venta (subtotal vs total, HMAC). TODO: emitir SALE_DISCOUNT explícito si supera umbral.
cambios de precio	' PRODUCT_UPDATE con { before, after }
cambios de inventario	' STOCK_AJUSTE, INV_MERMA, INV_REPOSICION
mermas	' INV_MERMA
cambios de rol	' USER_UPDATE (incluye rol)
cambios de permisos	' PERMISSION_UPDATE
acciones administrativas críticas	' ADMIN_PIN_SET, SETTINGS_UPDATE, OTP/HMAC reject

TODO / integración faltante (documentado, no implementado)

- Logout explícito: hoy el cierre de sesión es solo cliente. Sugerido: endpoint POST /api/auth/logout que invalide la clave HMAC en memoria y emita LOGOUT. (Stub: services/sessionKeys.js ya tiene el Map; agregar revokeSessionKey.)
- SALE_DISCOUNT explícito: emitir evento cuando el descuento supere un umbral configurable; enganchar con aprobación de Supervisor.
- Retención/expurgo: definir política de retención del log (hoy crece indefinidamente). Sugerido: export mensual + archivado.

- Hash encadenado (tamper-evidence fuerte): encadenar prev_hash por fila para detectar borrados a nivel de archivo. (Hoy la inmutabilidad es por trigger.)

Onboarding para desarrolladores

1. Requisitos

- Node.js 20+ y npm. Una base Turso (o un archivo libSQL local para desarrollo).

2. Puesta en marcha local

```
npm install # instala el monorepo

# .env del backend (apps/backend/.env)
# TURSO_DATABASE_URL=file:local-dev.db # o libsql://...turso.io
# TURSO_AUTH_TOKEN= # vacío si es archivo local
# JWT_SECRET=<cadena larga aleatoria>
# JWT_TTL=12h

# Esquema + datos base + usuario GERENCIA (imprime credenciales + OTP una vez)
node apps/backend/scripts/provision.mjs
node --env-file=apps/backend/.env apps/backend/scripts/set-admin-pin.mjs <PIN>

# correr (2 terminales)
npm run dev --workspace @cartel/backend # API en :3000
npm run dev --workspace @cartel/frontend # Vite en :5173 (proxy /api !' :3000)
```

> En producción Express sirve la build (apps/frontend/dist) en el mismo dominio;
> en dev el proxy de Vite enruta /api.

3. Scripts útiles (ver package.json)

Comando	Qué hace
npm run build	build del frontend (Vite)
npm start	inicia el backend (sirve API + dist)
npm test --workspace @cartel/backend	suite Vitest
node apps/backend/scripts/provision.mjs	aplica schema+seed y crea GERENCIA + OTP
scripts/set-admin-pin.mjs <PIN>	fija el PIN de administrador
scripts/backup.mjs	respaldo de la base
scripts/migrate-*.mjs	migraciones idempotentes (roles, permisos, admin-pin, etc.)

> Aplicar .sql: usa db.executeMultiple(sql) (parseo server-side, soporta
> triggers); no dividas por ; en cliente contra Turso remoto.

4. Convenciones de código

Frontend

- Sin react-router: App.jsx es un shell con estado screen; el menú y el guard por permiso salen de config/nav.js (fuente única de módulos). Las páginas públicas se resuelven por pathname en main.jsx.
- lib/api.js: cliente HTTP único (añade Authorization: Bearer y, si aplica, x-management-otp). Maneja 401 de sesión globalmente.
- lib/offlineStore.js: cola IndexedDB (ventas offline) + flushQueue().
- components/ui/States.jsx: Spinner / EmptyState / ErrorState / Forbidden / humanizeError. Toda pantalla con fetch debe tener los 4 estados (loading/error/empty/data).
- components/PeriodNav.jsx: navegación temporal (Día/Mes/Año) que emite {from,to} ISO.
- Marca/colores: tailwind.config.js (colors.cartel); textos de UI en español (Chile).

Backend

- Controladores delgados; lógica transversal en services/ (audit.writeAudit, permissions). Middlewares: requireAuth, requirePermission, requireOtpForMutation, verifyHmac, rateLimit.
- Auditar toda mutación relevante con writeAudit(...).
- Errores: res.status(code).json({ error: 'CODIGO' }) (el cliente los humaniza).

5. Flujo de trabajo (Git + deploy)

- Rama por cambio; PR a main. main auto-despliega a Render (autoDeploy: true).
- Mensajes de commit: tipo(scope): descripción (feat/fix/chore/docs), referenciando el ticket cuando aplique (ej. feat(KAN-22): ...). Co-autoría al final.
- No romper auth ni el layout; mantener los tests verdes antes de mergear.
- Verificar un deploy: el bundle servido cambia de hash (<https://cartel-pollos.onrender.com/> !' assets/index-XXXX.js).
- El plan free de Render duerme (~30-50s de arranque en frío). Tras un deploy, una pantalla ya abierta necesita Ctrl+Shift+R (caché del service worker PWA).

6. Tests

- Vitest en apps/backend/test (single-fork: BD en memoria + Map HMAC compartidos). Helpers en test/helpers.js (aplican schema, siembran usuarios, login, firma HMAC).
- Correr: npm test --workspace @cartel/backend. Mantener la suite verde.

7. Despliegue

- Render lee render.yaml. Variables secretas (TURSO_DATABASE_URL, TURSO_AUTH_TOKEN) se cargan en el dashboard; JWT_SECRET lo genera Render. Detalle en DEPLOY.md.
- Una base nueva se inicializa con provision.mjs (ver §2).

Notas de Migración — Refactor RBAC + Auditoría

1. Cambios potencialmente breaking

Cambio	Impacto	Mitigación
Anular venta: reports.view !' sales.void	Un rol que solo veía reportes ya no puede anular hasta tener sales.void.	migrate-perms.mjs siembra sales.void para SUPERVISOR/GERENCIA/ADMIN.
Nuevo permiso audit.view	Sin la fila en role_permissions, nadie ve Auditoría (ni gerencia).	migrate-perms.mjs lo siembra para GERENCIA/ADMIN.
Permisos de CAJERO reducidos (sin reports.view ni expenses.manage)	Un cajero existente pierde esos accesos (intencional, least-privilege).	Si se requiere, usar rol Supervisor o activar el permiso puntual en la matriz.
CHECK de role removido	Requiere reconstruir users y role_permissions en bases existentes.	migrate-roles.mjs (idempotente, preserva datos).
Frontend: shell/nav refactorizado	Cambian etiquetas/orden del menú; grupo Administración.	Sin breaking funcional; solo IA/labels.

> No se eliminó funcionalidad. Toda capacidad previa sigue disponible (con permisos reorganizados).

2. Supuestos

- La base de producción fue creada con el esquema antiguo (con CHECK de rol). Las migraciones detectan y solo actúan si corresponde (idempotentes).
- GERENCIA/ADMIN son superadmins: conservan permissions.manage por salvaguarda.
- El cliente libsql sobre archivo local mantiene PRAGMA foreign_keys=OFF durante el rebuild (verificado: integridad FK intacta tras migrar local).

3. Pasos de despliegue (orden importa)

> Ejecutar las migraciones antes o junto al deploy del código, para que no haya ventana sin permisos.

```
# 1) En producción (variables Turso en apps/backend/.env.production)
cd apps/backend

# 1a) Quitar el CHECK de rol (reconstruye users y role_permissions, preserva datos)
node --env-file=.env.production scripts/migrate-roles.mjs

# 1b) Sembrar permisos faltantes (6 roles + sales.void + audit.view)
node --env-file=.env.production scripts/migrate-perms.mjs

# 2) Subir el código (Render auto-deploy)
git push

# 3) Repetir 1a/1b en local si se va a operar local
node --env-file=.env scripts/migrate-roles.mjs
node --env-file=.env scripts/migrate-perms.mjs
```

Si migrate-roles.mjs reporta & Violaciones de FK, no continuar: revisar (no debería ocurrir; preserva ids).

4. Checklist de QA

- [] Login OK con gerencia; ver Administración !' Auditoría carga eventos.
- [] Crear usuario rol Supervisor, Despacho, Cocina, Administrador !' 201.
- [] Cajero (rol CAJERO): no ve Finanzas ni Administración; no puede anular.
- [] Supervisor: puede anular una venta (queda SALE_VOID en Auditoría).
- [] Ajuste de stock con PIN !' aparece STOCK_AJUSTE con stock anterior/nuevo.
- [] Cambiar un permiso en Roles y permisos !' aparece PERMISSION_UPDATE.
- [] Forzar pantalla sin permiso !' "Acceso restringido".
- [] Sesión: esperar inactividad / token vencido !' redirige a login con aviso.
- [] Vender con caja cerrada !' bloqueado con "Abrir caja".

5. Checklist de permisos (post-migración)

- [] GET /api/permissions devuelve 6 roles y role_meta.
- [] sales.void = sí para SUPERVISOR/GERENCIA/ADMIN; no para CAJERO/COCINA/DESPACHO.
- [] audit.view = sí solo para GERENCIA/ADMIN.
- [] permissions.manage = Ø=¿ sí para GERENCIA/ADMIN (no se puede apagar).

6. Checklist de rollout

- [] Migraciones corridas en prod (1a, 1b) sin warnings de FK.
- [] Deploy verificado: GET /api/audit responde a gerencia (200) y a cajero (403).
- [] Asignar roles reales a las cuentas (caja=CAJERO, cocina=PREPARADOR; crear Supervisor/Despacho según el equipo).
- [] (Opcional) Crear una cuenta ADMIN separada de la de dueño/a si se desea segregar sistema vs negocio.
- [] Comunicar al equipo el nuevo menú (Administración) y que anular ahora es permiso aparte.

7. Rollback

- El código es retrocompatible con datos: revertir el deploy (git) no requiere revertir las migraciones (las columnas/relajación de CHECK son aditivas/neutras).
- Si fuese necesario volver a 3 roles: basta reasignar usuarios a los 3 roles previos; las filas extra en role_permissions son inertes.
- La auditoría es append-only: no se borra; un rollback de código no afecta su contenido.

Addendum — Extracción de Código y Lógica de Negocio

Reporte técnico complementario para soporte continuo y escalamiento (operación San Ramón).
Contiene el código fuente exacto y la lógica subyacente de los módulos core.

1. Código Core — Seguridad y Persistencia

1.1 Triggers de inmutabilidad de auditoría (schema.sql)

La tabla audit_logs es append-only lógico: dos triggers BEFORE abortan cualquier UPDATE o DELETE posterior a la inserción. Como el INSERT de auditoría va en el mismo db.batch que la venta/cierre, un intento de tampering aborta toda la transacción (rollback).

```
CREATE TRIGGER IF NOT EXISTS audit_logs_no_update
BEFORE UPDATE ON audit_logs
BEGIN
    SELECT RAISE(ABORT, 'audit_logs es append-only: UPDATE no permitido');
END;

CREATE TRIGGER IF NOT EXISTS audit_logs_no_delete
BEFORE DELETE ON audit_logs
BEGIN
    SELECT RAISE(ABORT, 'audit_logs es append-only: DELETE no permitido');
END;
```

1.2 Firma HMAC-SHA256 (frontend lib/crypto.js)

La clave de sesión (hex 256-bit) se entrega en login, se importa como CryptoKey NO exportable (extractable=false) y vive solo en memoria. La canonicalización ordena claves recursivamente para coincidir byte a byte con el backend.

```
export async function setSessionKey(sessionId, hexKey) {
    _sessionId = sessionId; _sessionKey = hexKey;
    const raw = hexToBytes(hexKey);
    _cryptoKey = await crypto.subtle.importKey(
        'raw', raw, { name: 'HMAC', hash: 'SHA-256' }, false, ['sign']);
}

export function canonicalize(value) {
    if (Array.isArray(value)) return `[${value.map(canonicalize).join(',')}]`;
    if (value && typeof value === 'object') {
        const keys = Object.keys(value).sort();
        return `{${keys.map((k) => JSON.stringify(k) + ':' + canonicalize(value[k])).join(',')}`;
    }
    return JSON.stringify(value);
}

export async function signSale(payload) {
    if (!_cryptoKey) throw new Error('SESION_SIN_CLAVE');
    const data = new TextEncoder().encode(canonicalize(payload));
    const sig = await crypto.subtle.sign('HMAC', _cryptoKey, data);
    return { payload, sessionId: _sessionId, hash: bytesToHex(new Uint8Array(sig)) };
}
```

1.3 Verificación HMAC y rechazo de payloads (backend middleware/hmac.js)

Recalcula el HMAC con la clave de sesión del servidor y compara en tiempo constante (timingSafeEqual). Payload manipulado: 409 PAYLOAD_MANIPULADO + evento HMAC_REJECT (ALERT).

```
function safeEqual(a, b) {
  const ba = Buffer.from(a, 'hex'); const bb = Buffer.from(b, 'hex');
  if (ba.length !== bb.length) return false;
  return crypto.timingSafeEqual(ba, bb);
}
export async function verifyHmac(req, res, next) {
  const { payload, sessionId, hash } = req.body || {};
  if (!payload || !sessionId || !hash) return res.status(400).json({ error:
'FIRMA_INCOMPLETA' });
  const sessionKey = await getSessionKey(sessionId, req.user?.id);
  if (!sessionKey) return res.status(401).json({ error: 'SESION_NO_VALIDA' });
  const expected = crypto.createHmac('sha256', Buffer.from(sessionKey, 'hex'))
    .update(canonicalize(payload)).digest('hex');
  if (!safeEqual(expected, String(hash))) {
    await writeAudit({ userId: req.user?.id ?? null, action: 'HMAC_REJECT', entity: 'sales',
      entityId: payload.client_uuid ?? null, severity: 'ALERT', ip: req.ip, metadata:
{ sessionId } });
    return res.status(409).json({ error: 'PAYLOAD_MANIPULADO' });
  }
  req.verifiedPayload = payload; return next();
}
```

Wiring de la ruta (firma validada antes de la lógica de venta):

```
app.post('/api/sales/sync', requirePermission('pos.sell'), verifyHmac, syncSale);
```

1.4 Offline-First (frontend lib/offlineStore.js)

Cola en IndexedDB con keyPath client_uuid. La venta se FIRMA antes de encolar (no se puede alterar sin invalidar el HMAC). flushQueue() reintenta y trata 409 como éxito idempotente.

```
const DB_NAME = 'cartel-pollos'; const STORE = 'pending_sales'; const SYNC_URL = '/api/sales/
sync';
db.createObjectStore(STORE, { keyPath: 'client_uuid' });

export async function recordSale(sale) {
  const envelope = await signSale(sale);
  try {
    const res = await postSale(envelope);
    if (!res.ok) throw new Error('HTTP_' + res.status);
    const body = await res.json().catch(() => ({}));
    return { synced: true, order_number: body.order_number, total: body.total };
  } catch {
    await tx('readwrite', (s) => s.put({ ...envelope.payload, _envelope: envelope }));
    return { synced: false, queued: true };
  }
}
export async function flushQueue() {
  for (const item of await pendingList()) {
    try {
      const res = await postSale(item._envelope);
      if (res.ok || res.status === 409) await tx('readwrite', (s) =>
s.delete(item.client_uuid));
    } catch { /* sigue en cola */ }
  }
}
if (typeof window !== 'undefined') window.addEventListener('online', () => flushQueue());
```

Config PWA (vite.config.js): registerType autoUpdate; app-shell precacheado; catálogo/insumos NetworkFirst (timeout 4s, TTL 24h). Las mutaciones nunca se cachean:

```
navigateFallbackDenylist: [/^\/api\/$/, /^\/landing/],
```

2. Reglas de Negocio Estrictas

2.1 Combo Familiar — precios y variaciones

No hay lógica especial por producto. Un combo es una fila de products (category COMBO) con price base y receta BOM. Las variaciones provienen de modifier_options.price_delta. El price del POS se ignora: el backend relee product.price desde la DB. Fórmula real (services/sales.js):


```

const unitPrice = Number(product.price); // base del combo (DB)
const chosen = (line.modifier_option_ids || []).map((oid) => optionMap.get(oid)).filter(Boolean); // recargos desde DB
const modsTotal = chosen.reduce((s, o) => s + o.price_delta, 0);
const lineTotal = (unitPrice + modsTotal) * qty; // (base + recargos) x cantidad
const subtotal = total;
const discount = Math.min(Math.max(0, Number(rawDiscount) || 0), subtotal); // acotado [0, subtotal]
const deliveryFee = Math.max(0, Number(payload.delivery_fee) || 0);
total = Math.round((subtotal - discount + deliveryFee) * 100) / 100;

```

2.2 Validación anti-manipulación de modificadores

El POS solo envía IDs de opciones; el backend resuelve nombre y price_delta desde la tabla, solo opciones is_active=1. El JSON modifiers persistido se reconstruye desde la DB, no del cliente.

```

const allOptionIds = [...new Set(items.flatMap((i) => Array.isArray(i.modifier_option_ids) ? i.modifier_option_ids : []))];
const optRes = await db.execute({
  sql: `SELECT id, name, price_delta FROM modifier_options WHERE is_active = 1 AND id IN (${allOptionIds.map(() => '?').join(',')})`,
  args: allOptionIds });
for (const o of optRes.rows) optionMap.set(o.id, { name: o.name, price_delta: Number(o.price_delta) });

```

2.3 Congelamiento de costo en inventory_adjustments (tipo VENTA)

El descuento BOM escribe un ajuste VENTA con el cost_unit vigente al momento. El P&L histórico no cambia aunque el costo del insumo se actualice después. Inventario teórico estricto por BOM, sin ensaladas ni vegetales frescos (regla de negocio).

```

for (const [ingId, use] of ingredientUse) {
  stmts.push({ sql: `UPDATE ingredients SET stock_qty = stock_qty - ?, updated_at = datetime('now') WHERE id = ?`, args: [use.qty, ingId] });
  stmts.push({ sql: `INSERT INTO inventory_adjustments (id, ingredient_id, user_id, type, qty_delta, unit_cost, reason, sale_id) VALUES (?, ?, ?, 'VENTA', ?, ?, ?, ?)`, args: [randomUUID(), ingId, ctx.userId, -use.qty, use.cost, // use.cost = costo congelado `Descuento BOM por venta ${client_uuid}`, saleId] });
}

```

El COGS del P&L usa ese costo: SUM(ABS(qty_delta) * unit_cost) WHERE type IN ('VENTA','MERMA'). La verificación de stock ocurre antes de comprometer; todo corre en un db.batch atómico con rollback.

3. Hardware e Integraciones

3.1 Tickets térmicos 58/80mm (lib/receipt.js)

Ancho desde business_settings.paper_width (CHECK 58/80). @page define tamaño de rollo y margen 2mm; fuentes monoespaciadas escaladas por ancho. El documento se autoimprime y se cierra.

```

function docShell(width, inner) {
  const w = width === 58 ? 58 : 80;
  return `<!doctype html><html><head><meta charset="utf-8">
<style>
  @page { size: ${w}mm auto; margin: 2mm; }
  * { box-sizing: border-box; }
  body { width: ${w}mm; margin: 0; font-family: 'Courier New', monospace; color: #000; }
  .r { font-size: ${w === 58 ? 11 : 12}px; line-height: 1.35; }
  .big { font-size: ${w === 58 ? 16 : 19}px; font-weight: 700; }
  .xl { font-size: ${w === 58 ? 30 : 38}px; font-weight: 800; }
  hr { border: none; border-top: 1px dashed #000; margin: 4px 0; }
</style></head><body><div class="r">${inner}</div>
<script>window.onload=function(){window.print();setTimeout(function(){
window.close();},300);};</script>
</body></html>`;
}

```

Mecanismo (lib/print.js): no usa ESC/POS. Abre ventana, inyecta HTML y dispara window.print(). Funciona con cualquier impresora térmica instalada como impresora del SO.

3.2 WhatsApp — solo wa.me en cliente, sin backend

Confirmado: no hay integración de servidor (ni WhatsApp Business API ni webhooks). Solo enlaces wa.me generados en el cliente; el envío lo confirma un humano.

```
export function whatsappUrl(data, settings = {}, phone) {  
  const text = encodeURIComponent(buildWhatsappText(data, settings));  
  const to = (phone || data.client_phone || '').replace(/[\^d]/g, '');  
  return to ? `https://wa.me/${to}?text=${text}` : `https://wa.me/?text=${text}`;  
}
```

El otro uso de wa.me es el QR de la cartelería pública (PublicCartelera.jsx) hacia <https://wa.me/<whatsapp>> para pedidos. El número sale de business_settings.whatsapp.

4. Estado Actual y Deuda Técnica

4.1 Bugs / comportamientos conocidos

Estado	Detalle
Resuelto	Gráficos del Resumen vacíos: items-end impedía estirar columnas; fix h-full.
Resuelto	HTTP 503 en /reports/turnos: usaba u.name; la columna real es users.full_name.
Resuelto	Edición de costo bloqueada por OTP en no-gerencia; reenrutada a set-stock (PIN).
Menor	Git advierte LF->CRLF en Windows al commitear (no afecta runtime).
Contenido	Foto cartelería colaciones con fondo claro (difiere de hero-*.jpg oscuras).
Por diseño	restock actualiza cost_unit por promedio ponderado (P&L histórico intacto).

No hay bugs abiertos críticos. Suite backend en 123/123 verde.

4.2 Backlog técnico / módulos inconclusos

Prioridad	Item
BLOQUEANTE	Multi-local/multi-tenant para San Ramon: business_settings es fila unica (id=1) y solo una caja abierta global (uniq_one_open_session). Operar 2 sucursales exige un store_id en users, sales, cash_sessions, inventario, settings, reportes y RBAC.
Alta	Replicar features a mi-rotiseria-pos (white-label): prompt generado, no ejecutado.
Media	conteo_umbral solo editable por SQL; exponerlo en Negocio/ Ajustes.
Media	WhatsApp manual: sin envio automatico ni confirmacion de entrega.
Media	Impresion via dialogo del SO (window.print); sin ESC/POS directo.
Media	Sin tests automatizados de frontend (solo vitest backend).
Baja	Prediccion de horno: fetch de clima best-effort, se omite si falla.
Baja	RBAC de modulos nuevos (KDS, Cuadre, Consumo, Precios) a revisar al introducir store_id.

Resumen para el equipo entrante

- Seguridad: JWT + HMAC por sesion en memoria + audit_logs inmutable por triggers + PIN admin (bcrypt) + TOTP.
- Integridad de precios: el backend es la unica fuente de verdad (precio, recargos, descuento acotado, costo BOM congelado). El POS no puede inflar montos.
- Resiliencia: offline-first con cola IndexedDB firmada + idempotencia por client_uuid.
- Mayor riesgo de escalamiento: arquitectura mono-local; San Ramon exige multi-sucursal antes de operar en paralelo.

